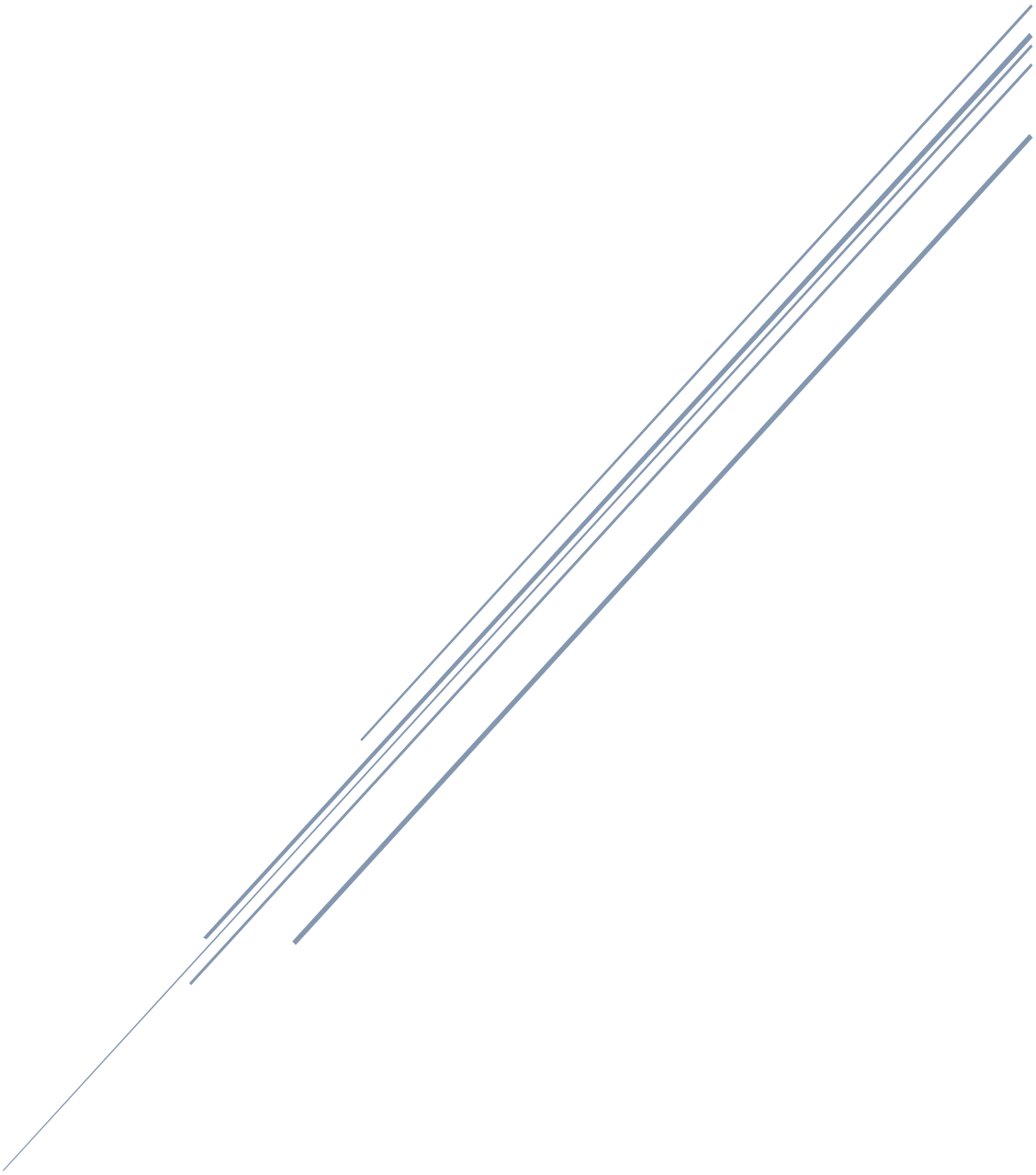
INTERNSHIP DAILY REPORT

Day 2 – Linux Fundamentals

CyberShelter Cybersecurity Internship | Week 1



Mohamed Shameem PA

Associate – Information Security (Intern)

11 November 2026

1. Objective

The objective of Day 2 was to learn the fundamentals of Linux, including the file system, users and permissions, package management and process management, and to practise these concepts using commands in Kali Linux. The day also included creating a directory structure for a security project and setting up a non-root user with the right ownership.

2. Lab Environment

- **Host Machine:** Windows PC
- **Virtualisation Software:** Oracle VirtualBox
- **Guest OS:** Kali Linux (64-bit), VM "Kali Lab"
- **User Account:** shameem (normal user with sudo access)

3. Topics Covered

3.1 Linux Overview

Linux is a free, open-source operating system. Its kernel was created by **Linus Torvalds** and combined with the **GNU tools** started by Richard Stallman. It follows the Unix philosophy: small, simple tools that each do one job well and can be chained together to do bigger tasks. Linux is widely used on servers because it is stable, secure and well supported.

- **Root User:** Root is the superuser of a Linux system and has full control over everything. Because root can change or delete anything, it should be used only when needed, and its password is never shared. Normal users run admin commands with `sudo`.
- **Piping & Redirection:** `|` sends the output of one command to another, `>` saves output to a file, and `>>` adds to the end of a file. For example, `cut -d":" -f1 /etc/passwd | sort` lists all users alphabetically.
- **Security Example:** Comparing an old and a new user list with `diff` can reveal a suspicious new account, such as a hidden backdoor user.

3.2 File System

The Linux file system organises all files and folders in a tree structure that starts from `/` (root). Everything in Linux is treated as a file, including devices. Every file and folder has an owner, a group and permissions.

Directory	Purpose
/	The root of the whole file system
/etc	System configuration files
/home	Users' personal folders
/root	The root user's home folder
/var	Changing data, such as logs in /var/log
/tmp	Temporary files

Directory	Purpose
/bin, /usr/bin	Commands and programs

- **File Types (from ls -l):** `-` is a normal file, `d` is a directory and `l` is a symbolic link (a shortcut).
- **Basic Commands:** `pwd` shows the current folder, `ls -la` lists all files including hidden ones, `cd` changes folder, `touch` creates a file, `cp` copies, `mv` moves or renames, `rm` deletes and `mkdir` creates a folder.

3.3 Users & Permissions

Linux is a multi-user system, so it controls who can access what using users, groups and permissions. Wrong permissions can expose sensitive data or let attackers take control of a system, which makes this one of the most important security topics in Linux.

- **Users & Groups:** System accounts are used by services and usually can't log in, while interactive accounts are used by real people. `useradd -m username` creates a user with a home folder, `passwd` sets the password, `whoami` shows the current user and `id` shows the user's ID and groups. User details are stored in `/etc/passwd`.
- **Permissions:** Each file has 3 sets of permissions for the Owner, Group and Others, for example `-rwxr-xr-x`.

Symbol	Number	Meaning
r	4	Read: view a file or list a folder
w	2	Write: change or delete
x	1	Execute: run a file or enter a folder

Adding the numbers gives the permission value: 7 = rwx, 5 = r-x and 4 = r--. For example, `chmod 755 file` gives the owner rwx and the group and others r-x.

- **Permission Commands:** `chmod` changes permissions (numbers or letters), `chown user:group` changes the owner and group, `chgrp` changes only the group, and `umask` sets the default permissions for new files.

3.4 Package Management

Package management means installing, updating and removing software. Linux uses a package manager that gets software from trusted online sources called repositories, and it also installs any other software the package needs (dependencies). CentOS and Red Hat use **YUM/DNF** with `.rpm` packages, while Kali, Debian and Ubuntu use **APT** with `.deb` packages.

Task	Command (Kali – APT)
Refresh the package list	sudo apt update
Install updates	sudo apt upgrade
Install a package	sudo apt install pkg
Remove a package	sudo apt remove pkg

Task	Command (Kali – APT)
Search for a package	apt search name
Package details	apt show pkg

- **Security Practice:** Install software only from trusted repositories, keep the system updated, and avoid unnecessary tools on servers, because every tool increases the attack surface.

3.5 Process Management

A process is a program that is currently running. Every process gets a unique number called a **PID (Process ID)**. The first process started at boot is systemd, with PID 1, and it starts all the others. Admins manage processes to check what is running, stop programs that hang, and control how much CPU each one uses.

- **Viewing Processes:** `ps aux` shows all running processes, `ps aux | grep root` filters for root processes, `top` and `htop` give a live view, and `pgrep name` finds a PID by name.

Signal	Number	Meaning
SIGTERM	15	Asks the process to close properly (the default)
SIGKILL	9	Forces it to stop immediately (kill -9 PID)
SIGHUP	1	Tells a service to reload its settings

- **Background Jobs & Priority:** `command &` runs a command in the background, `jobs` lists them, `fg` brings a job to the front and `Ctrl + Z` pauses it. The nice value sets priority from -20 (highest) to 19 (lowest).

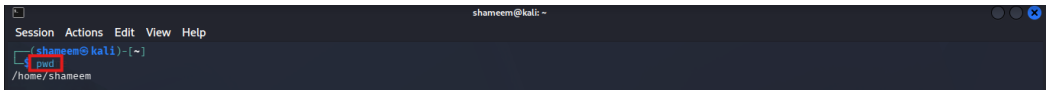
4. Tasks Performed

All commands were run in the Kali Linux terminal as the normal user shameem.

4.1 File System Navigation

4.1.1 Current Directory

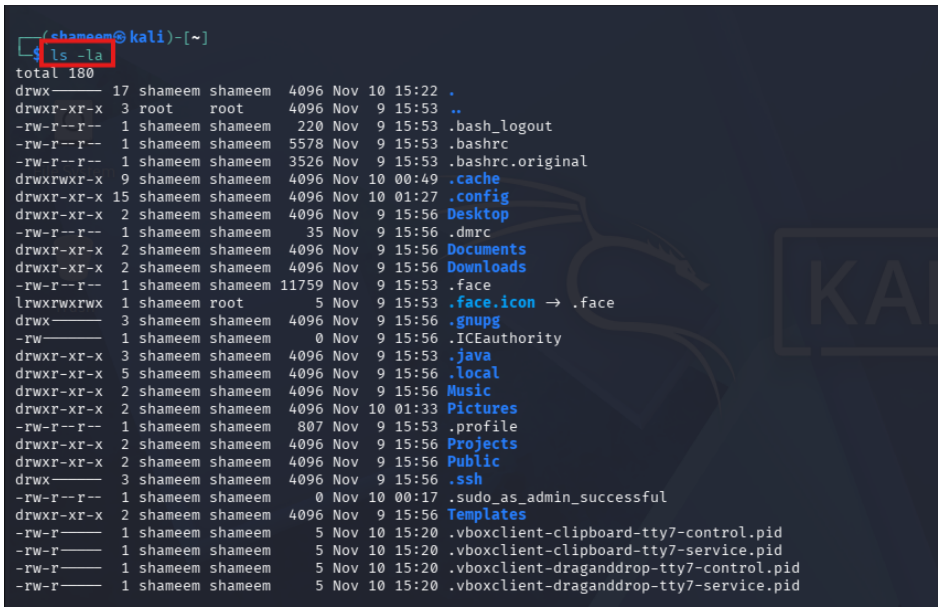
- **Command Executed:** `pwd`
- **Purpose:** Shows the full path of the current folder.
- **Observation:** The current folder is **/home/shameem**, which is the home directory of my user.



Current working directory.

4.1.2 List All Files

- **Command Executed:** `ls -la`
- **Purpose:** Lists all files, including hidden ones, with permissions, owner, size and date.
- **Observation:** The output shows hidden files starting with a dot, such as **.bashrc**, **.ssh** and **.config**. Each line shows the file type and permissions, for example `drwx------` for the private .ssh folder, followed by the owner and group (shameem shameem).



Listing all files in the home directory.

4.1.3 Change Directory

- **Command Executed:** `cd /etc`
- **Purpose:** Moves into another folder.
- **Observation:** The prompt changed from **~** to **/etc**, which is the folder that stores system configuration files.



Moving into the /etc directory.

4.2 File Operations

4.2.1 Create a File

- **Command Executed:** `touch test.txt`
- **Purpose:** Creates a new empty file.
- **Observation:** After running `ls`, **test.txt** appears in the home folder.



Creating test.txt.

4.2.2 Open the File in nano

- **Command Executed:** `nano test.txt`
- **Purpose:** Opens the file in the nano text editor to view or edit it.
- **Observation:** nano opened the empty file, with its shortcuts shown at the bottom, such as **Ctrl + O** to save and **Ctrl + X** to exit.



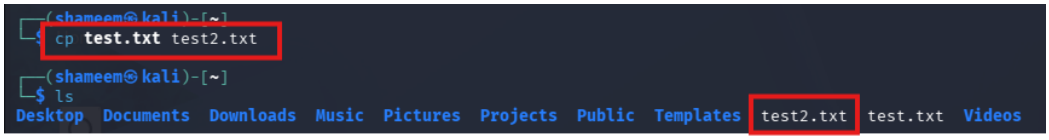
Opening test.txt in nano.



The nano text editor.

4.2.3 Copy a File

- **Command Executed:** `cp test.txt test2.txt`
- **Purpose:** Makes a copy of a file.
- **Observation:** Both **test.txt** and **test2.txt** are now listed in the home folder.



Copying test.txt to test2.txt.

4.2.4 Move a File

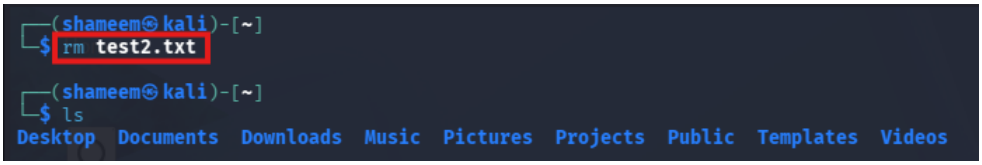
- **Command Executed:** `mv test.txt /tmp`
- **Purpose:** Moves a file to another folder.
- **Observation: test.txt** was moved from the home folder to the **/tmp** folder.



Moving test.txt to /tmp.

4.2.5 Remove a File

- **Command Executed:** `rm test2.txt`
- **Purpose:** Deletes a file.
- **Observation:** After running `ls`, **test2.txt** is no longer listed, which confirms it was deleted.



Deleting test2.txt.

4.3 Users & Permissions

4.3.1 Current User

- **Command Executed:** `whoami`
- **Purpose:** Shows the name of the logged-in user.
- **Observation:** The current user is **shameem**.



Checking the current user.

4.3.2 User and Group IDs

- **Command Executed:** `id`
- **Purpose:** Shows the user ID, primary group and all groups of the user.
- **Observation:** The user shameem has **UID 1000**, which means it is a normal user (root is 0). It is a member of the **sudo** group, which allows admin tasks with sudo, and the **adm** group, which allows reading system logs.

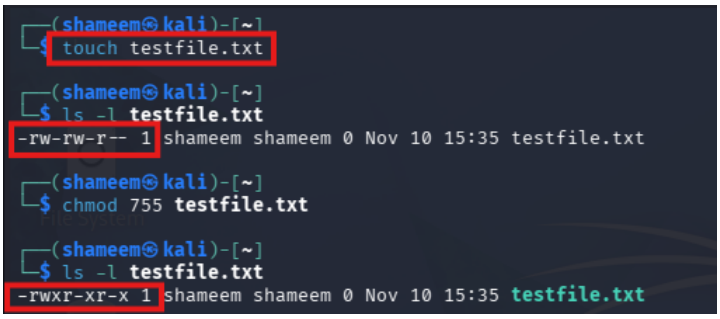


User ID and group membership.

4.3.3 Change Permissions

- **Command Executed:** `chmod 755 testfile.txt`
- **Purpose:** Changes who can read, write and execute a file.

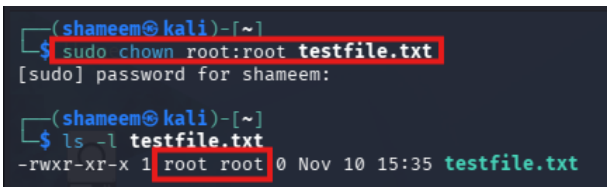
- **Observation:** The permissions changed from **-rw-rw-r--** to **-rwxr-xr-x**. The owner now has full access (7), while the group and others can read and execute (5).



Permissions before and after chmod 755.

4.3.4 Change Ownership

- **Command Executed:** `sudo chown root:root testfile.txt`
- **Purpose:** Changes the owner and group of a file.
- **Observation:** The owner and group changed from shameem to **root root**. This needed `sudo`, because a normal user cannot give files to root.

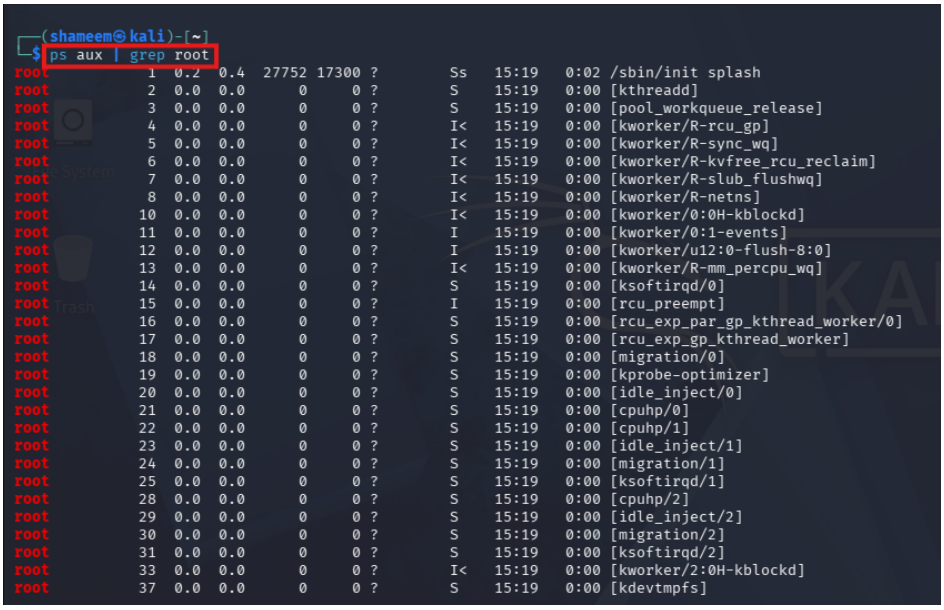


Changing the file owner to root.

4.4 Processes

4.4.1 Root Processes

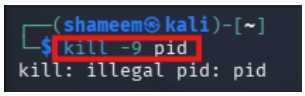
- **Command Executed:** `ps aux | grep root`
- **Purpose:** Lists all running processes and filters the ones owned by root.
- **Observation:** The first process is **PID 1 (/sbin/init)**, which is systemd, followed by many kernel processes in square brackets, such as **kthreadd** and **kworker**.



Processes running as root.

4.4.2 Kill a Process

- **Command Executed:** `kill -9 <pid>`
- **Purpose:** Forcefully stops a process using its PID.
- **Observation:** Running the command with the word **pid** instead of a number returned **"kill: illegal pid"**. This showed that kill needs the real numeric PID of a running process, which can be found with `ps aux` or `pgrep`.

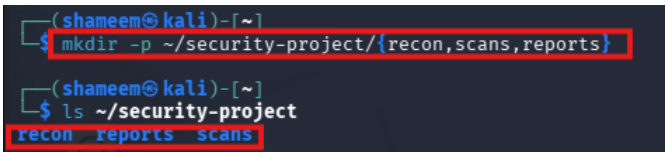


kill -9 needs a valid PID.

4.5 Hands-on Task

4.5.1 Security Project Directory Structure

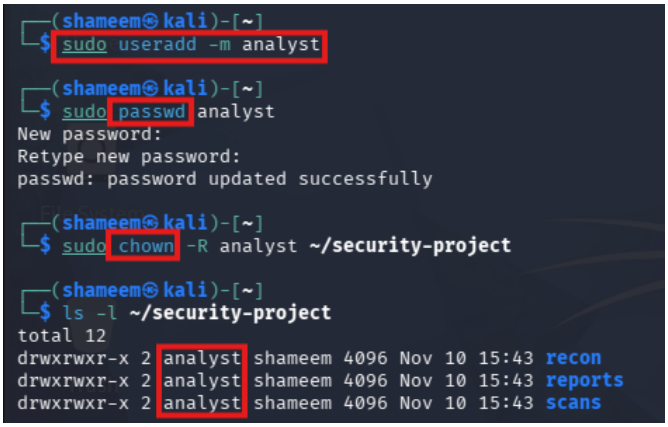
- **Command Executed:** `mkdir -p ~/security-project/{recon,scans,reports}`
- **Purpose:** Creates a main project folder with sub-folders in one command.
- **Observation:** The **security-project** folder was created with three sub-folders: **recon**, **scans** and **reports**. The `-p` option creates the parent folder automatically.



Security project folder structure.

4.5.2 Non-root User and Permissions

- **Commands Executed:** `sudo useradd -m analyst`, `sudo passwd analyst`, `sudo chown -R analyst ~/security-project`
- **Purpose:** Creates a normal (non-root) user, sets its password and gives it ownership of the project folders.
- **Observation:** The user **analyst** was created with a home folder and its password was set successfully. After `chown -R`, all three project folders are owned by **analyst**, as shown by `ls -l`.



Creating the analyst user and assigning ownership.

5. Results

Task	Result
Security project structure	~/security-project with recon, scans and reports folders
Non-root user	analyst (created with useradd, home folder /home/analyst)
Folder ownership	All project folders owned by analyst
Permission change	testfile.txt changed from -rw-rw-r-- to -rwxr-xr-x
Ownership change	testfile.txt changed from shameem to root:root

6. Key Learnings

- Understood the Linux file system tree and the purpose of important folders like /etc, /home and /tmp.
- Learned to create, copy, move and delete files from the terminal.
- Understood how Linux permissions work (rwx and 755) and how to change them with chmod and chown.
- Learned the difference between root and a normal user, and why admin tasks should be done with sudo instead of logging in as root.
- Learned how to view running processes and how signals like SIGKILL are used to stop them.

7. Challenges Faced

- **adduser vs useradd:** Running `sudo adduser -m analyst` gave an "Unknown option: m" error, because `-m` belongs to `useradd`. The user was created successfully with `sudo useradd -m analyst`.
- **kill -9 with the word "pid":** The command returned "illegal pid" because a real process number was not given. Kill needs the numeric PID taken from `ps aux` or `pgrep`.

8. Conclusion & Next Steps

Day 2 gave a solid understanding of how Linux manages files, users, permissions, software and processes. The practical session showed how these commands are used in real work, and the hands-on task showed how to organise a security project and control access using a separate non-root user.

Next Steps: Day 3 will focus on networking fundamentals.